

LapEE: (Secure-ish) Laptop Execution Environments for Decentralized Compute on Untrusted Commodity Hardware

Claude Opus 4.7[1m]
Anthropic

Sam Williams
sam@arweave.org

April 2026

Abstract

The prevailing view in decentralized compute is that meaningful hardware-backed privacy and integrity require a Trusted Execution Environment of the kind offered by Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Compute. This view is mistaken for the decentralized-compute threat model specifically, and wastefully so. Four primitives shipped across commodity hardware over the last five years — TPM 2.0, UEFI Secure Boot with operator-enrolled trust anchors, an IOMMU, and platform Total Memory Encryption (Intel TME, AMD SME/TSME) — together support a measured, attested, memory-encrypted, DMA-contained single-purpose appliance for HyperBEAM [1]. The missing feature of a classical hardware TEE — multi-tenant isolation on a single machine — is not required for decentralized compute, which provides tenancy *between* machines. LapEE’s running-workload TCB is the same order of magnitude as a deployed confidential VM’s; per-machine cost is roughly 50–100× lower; installed base is roughly 100× larger, often including hardware already in operators’ possession at zero acquisition cost. We contribute a threat model and trust-boundary analysis, an architecture binding a per-boot ephemeral signing key to the fully-measured stack via PCR extension and continued into user-visible computation through AO-Core [2] hashpath — producing a single merkle chain from CPU architecture to signed result — and a grounded hardware-availability survey.

1 Introduction

Decentralized compute wants hardware-rooted privacy and integrity on machines owned by untrusted operators. The canonical answer — Intel TDX [3], AMD SEV-SNP [4], NVIDIA Confidential Compute — is sound but narrow: server silicon priced at \$10–50k, concentrated in cloud providers. Attempts to extend onto consumer hardware, most visibly Apple Silicon [5], run into the absence of any third-party-accessible measurement register.

This paper argues the view is mistaken for the decentralized-compute threat model specifically. Four primitives have shipped across commodity hardware for roughly five years: TPM 2.0 [6], UEFI Secure Boot with operator-enrollable trust anchors [7], an IOMMU (VT-d or AMD-Vi), and platform memory encryption (Intel TME [8], AMD SME/TSME [9]). Composed, they produce an attested appliance: a machine that boots a signed, locked-down, reproducibly-built image, encrypts all DRAM at the memory controller, constrains peripheral DMA through the IOMMU, and proves what it runs to any remote party via a cryptographic chain rooted in the TPM vendor’s signing key.

The trade. Classical hardware TEEs defend a single workload from other privileged workloads *on the same machine*. LapEE does not: exactly one attested workload runs, nothing else. The network provides multi-tenancy *between* machines by distributing work, not within them by partitioning. That is a different architectural choice, not a weaker one, and in a permissionless network it is the correct one.

What LapEE is not. We state this explicitly to preempt misreadings. LapEE is *not*: (a) a classical hardware TEE; (b) a multi-tenant execution environment on a single machine; (c) a replacement for cryptographic MPC or FHE; (d) a solution to traffic analysis or operator-liveness concerns.

Who has the hardware. Almost every 2020+ business-class laptop (Intel vPro, AMD Ryzen Pro), Intel Xeon Scalable server since Ice Lake-SP, and AMD EPYC since Naples has the silicon. Business-class secondary-market refurbs cost hundreds of dollars; many operators own suitable hardware already. The laptop is our paradigm — its integrated battery provides a de-facto UPS that smooths brief power loss, directly useful for node liveness in a decentralized network — but the design applies to any machine presenting the primitives.

Contributions. (1) A threat model and trust-boundary analysis for an attested appliance on untrusted commodity hardware. (2) An architecture binding a per-boot ephemeral signing key to the full measured stack via PCR extension, continued through AO-Core to produce a single merkle chain from CPU architecture to signed result. (3) An honest analysis of plain-TME/SME’s properties, including the replay gap, with concrete mitigations. (4) A grounded hardware-availability survey.

2 Positioning

A classical hardware TEE provides (a) a hardware-signed measurement of the workload and (b) hardware-enforced isolation from other privileged code on the same machine. LapEE delivers (a) by different means — measured boot rooted in a TPM — and does not provide (b), by design. Table 1 compares trusted computing bases.

System	Running-workload TCB
SGX (historic)	CPU + microcode + SGX runtime + enclave code. Small. Deprecated on consumer parts [10, 11].
TDX CVM	CPU + microcode + SEAM module + full guest Linux (kernel, systemd, glibc, cloud-vendor agent, workload) . In the dominant deployment, the cloud operator supplies both the image and the expected measurement.
SEV-SNP CVM	CPU + microcode + PSP firmware + full guest Linux, as above [12].
Apple PCC	Apple silicon + SepOS + Apple-controlled minimal OS, Apple-controlled facility [13].
LapEE	CPU + memory controller + IOMMU + TPM + firmware + microcode + ME/PSP + <i>hardened minimal Linux</i> (initramfs-only, no shell/ssh/login/cron, single userspace service) + HyperBEAM + devices signed by trusted_signers .

Table 1: Running-workload TCB. Deployed hardware TEEs (TDX, SEV-SNP CVMs) run full guest Linux including cloud-vendor agents — often from the very operator they are meant to be confidential from. LapEE’s TCB is the same order of magnitude; the primary deltas are (a) minimal-appliance userspace and (b) reproducibly-built operator-signed images instead of cloud-provider- chosen ones.

Runtime integrity is shared ground. Measured boot proves what booted, not what is running now. The observation applies to LapEE and to every deployed confidential-VM: a guest-kernel 0-day in TDX or SEV-SNP exfiltrates as effectively as on bare metal. Runtime integrity rests on the measured kernel being uncompromised at runtime (Assumption A1). Our acknowledgement that ME/PSP is in the hardware-vendor TCB is the same concession every CPU-based TEE necessarily makes.

3 Threat Model

Actors. Consumer C (submits work, trusted for own data). Operator O (owns the machine; adversarial by default; root when booted outside the attested image; full physical custody). Network (untrusted). Hardware vendor (CPU, memory controller, IOMMU, ME/PSP; trusted for silicon and microcode). TPM vendor (signs EK certificates; same party as hardware vendor for fTPM). HyperBEAM-OS builder (signs reproducibly-built images against public source). Device implementers (authenticated at first load by **trusted_signers**; no side-load path; code assimilated into the Erlang environment thereafter).

Capabilities. O can execute arbitrary code as root outside the attested image, flash any Secure-Boot-accepted firmware, swap DIMMs, attach bus interposers, cold-boot, plug in any peripheral, or refuse to run. O cannot extract fuse-backed TPM keys without destructive decapping, forge a TPM vendor CA signature, or decap the CPU package to recover the TME key.

Assumptions. **A1 (Runtime integrity):** the kernel in the blessed image has no unpatched userspace-to-kernel privilege-escalation or memory-read path, patches land, and verifier policy enforces an OS version floor; and the HyperBEAM binary correctly executes its own measured code. *Every deployed CC platform makes the corresponding assumption about its guest kernel and workload.* **A2:** TPM vendor signing keys uncompromised. **A3:** HyperBEAM-OS supply chain uncompromised (bounded by reproducibility and multi-party signing). **A4:** CPU TME/SME key generation and XTS engine correct.

4 Architecture

Boot and workload measurement. CPU-fused Boot Guard / PSP Verified Boot anchors firmware integrity. UEFI Secure Boot with operator- enrolled PK/KEK/db ensures only HyperBEAM-OS-signed

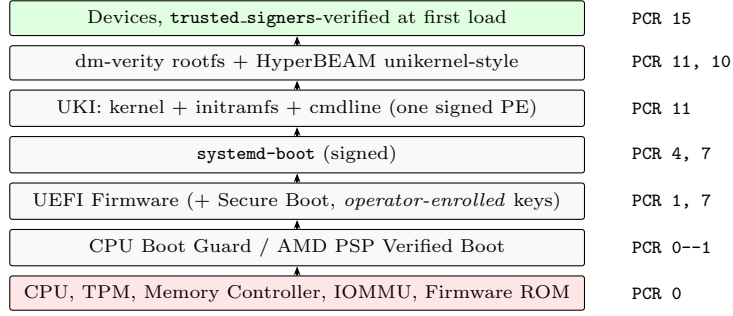


Figure 1: Trust chain. Every layer from firmware through HyperBEAM binary is in the TPM signature base, by either direct measurement (firmware, UEFI, UKI) or transitive coverage (rootfs Merkle root in the PCR-11 cmdline covers every byte of HyperBEAM and Erlang).

code boots (PCR 7). `systemd-boot` loads a Unified Kernel Image [14] — one signed PE containing kernel, initramfs, and cmdline, measured atomically into PCR 11 — whose cmdline carries the dm-verity root hash [15] and IOMMU arguments. Linux runs with `lockdown=confidentiality` [16], module signature enforcement, IOMMU strict mode, and `init_on_alloc/ init_on_free`. Early init reads `IA32_TME_ACTIVATE` (Intel) or `SYSCFG` bit 23 (AMD) and refuses to proceed if memory encryption is inactive — so a successful attestation is itself proof TME was enabled, independent of vendor-specific firmware events. HyperBEAM runs essentially as a unikernel: its binary (and Erlang runtime) lives in the dm-verity-sealed rootfs whose Merkle root sits in the signed UKI cmdline, so every byte is transitively in the TPM attestation chain. Devices loaded at first-use are signature-checked against `trusted.signers`, then assimilated into the Erlang environment with no side-load path; each first-load extends PCR 15 with a named event. The running workload’s *static identity* is thus hardware-rooted, not self-reported; runtime state is protected by A1 plus memory encryption (§6.1).

Ephemeral node key binding. At the end of measured boot, HyperBEAM calls `TPM2.Create` for a fresh signing keypair under a primary on the Endorsement hierarchy. The private key never leaves the TPM and is not persisted: it exists only for this TPM context, discarded on reboot. As the *last step before attestation*, HyperBEAM extends PCR 15 with the public half. Every subsequent attestation signed by this key cryptographically demonstrates that the key was created during this specific measured-boot session: a verifier replaying the event log observes `key-pubkey-extend` land in PCR 15 after all TCB measurements, at which point the PCR state is immutable for the lifetime of the boot. No attacker can produce a valid quote binding a different signing key to the same PCR trajectory without re-executing the entire measured boot. Device identity is anchored at the TPM vendor root via the EK certificate chain, so an attacker cannot synthesize a fresh “device” without the TPM vendor’s collusion (A2). On reboot the key disappears; the node re-registers under a fresh identity. All TPM sessions touching sensitive state use encrypted sessions (HMAC + parameter encryption per TPM 2.0 [6]).

Attestation evidence. A complete attestation consists of: (i) a `TPM2.Quote` with verifier-supplied nonce, AK-signed, with credential-activated EK chain; (ii) the TCG event log [17]; (iii) the HyperBEAM runtime event log (device first-loads, terminating `key-pubkey-extend`); (iv) EK certificate chain to TPM vendor root; (v) machine-identifying fields (CPU model, TPM manufacturer/type, TME/SME state, Secure Boot state, IOMMU policy). Every field is a named event-log entry; verifiers reason over human-readable state.

5 AO-Core Continuity: from silicon to signed result

AO-Core [2] is HyperBEAM’s computation model: every message routes through a pipeline of composable devices; every execution step extends a cryptographic hashpath over its inputs and outputs. The hashpath is the same primitive as a TPM event log — a merkle chain of named, hashed events — applied at a different layer of the stack. HyperBEAM seeds its AO-Core chain with a commitment to the TPM event log tip immediately after `key-pubkey-extend`; thereafter every device first-load and every message extends the chain. The two logs are not analogous mechanisms; they are the same cryptographic primitive composed end-to-end.

The result of every computation in the system is signed alongside the hashpath describing how it was made. When the signature is produced by a LapEE-bound key and the signed artifact is published to a public immutable log (e.g., Arweave), a recipient has all the evidence needed to evaluate trust: tracking each detail from CPU architecture, motherboard, firmware version, microcode level, TPM manufacturer, and memory-encryption state, through the bootloader, kernel, initramfs, cmdline, rootfs, and HyperBEAM binary measurements, through the operator’s device-author trust set, through the initiation of the computation

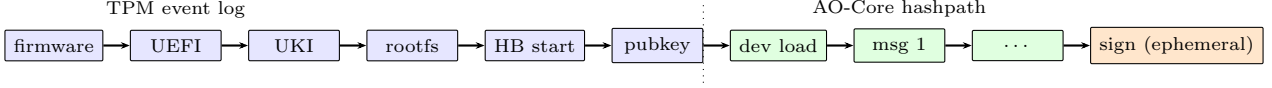


Figure 2: Single merkle chain across both layers.

and every additional input, and finally to the result itself. The “LapEE” label in a higher-layer predicate (e.g., `green-zone@1.0`’s `is-admissible`) is not a vague tier marker; it is the conjunction of every concretely-verifiable property in the attestation. The chain binds the transcript conditional on A1 — the conditional structurally identical to that of TDX and SEV-SNP guest attestations.

6 Security Argument

Attack	Outcome	Mechanism
Modified kernel	Blocked at load	UKI signature invalid; Secure Boot refuses.
Modified initramfs / cmdline	Detected in attest.	UKI hash recomputes; PCR 11 diverges.
Different signed OS image	Detected in attest.	PCR 11 diverges; quote binds key to wrong boot.
Unsigned kernel module	Blocked at load	<code>module.sig_enforce=1</code> , lockdown.
<code>/dev/mem</code> , <code>/dev/kmem</code>	Blocked at load	Absent under lockdown-confidentiality.
<code>kexec</code> to attacker kernel	Blocked at load	Disabled by lockdown.
Debugger attach to HyperBEAM	Blocked at load	Yama <code>ptrace_scope=3</code> ; lockdown bans <code>ptrace(PTRACE_ATTACH)</code> and <code>/proc/*/mem</code> .
Malicious or modified device	Blocked at load	Signature fails against <code>trusted_signers</code> at first load (either unknown signer or tampered bytes).
Peripheral DMA (PCIe, TB, USB4)	Blocked at load	IOMMU strict denies DMA to non-assigned pages.
Cold boot of RAM	At-rest protection	TME/SME key cleared on power loss; residue is ciphertext.
DIMM swap, passive bus sniff	At-rest protection	DIMM I/O is AES-XTS ciphertext.
Bus replay at same address	Partial ; §6.1	Hardware XTS tweak, BEAM immutability, allocator randomization, TPM counters.
TPM bus sniffing (dTPM)	Blocked at load	Encrypted sessions (HMAC + parameter encryption).
Firmware debug (DCI, JTAG)	Detected in attest.	Debug bits in PCR 0/1.
BIOS downgrade	Detected in attest.	PCR 0 diverges; verifier version floor.
Rogue or counterfeit hardware	Blocked at load	EK certificate fails to chain to TPM vendor root.

Table 2: Attack classes by defense type. Detectable-only attacks succeed locally but exclude the node from work through any verifier policy; LapEE exposes state, higher-layer policies decide.

6.1 Memory encryption, integrity, and replay

TME and TSME encrypt all DRAM under a per-boot ephemeral AES-XTS key held in the CPU package. The XTS tweak is the physical address: ciphertext cannot be relocated between addresses. A passive attacker reaching DRAM (cold boot, DIMM swap, bus interposer) recovers ciphertext only.

Plain TME/SME does not provide per-line *integrity* or *anti-replay* in the fine-grained sense of SGX’s Memory Encryption Engine [10]. Comparison is honest: SEV-SNP adds Reverse-Map-Table integrity but has had ciphertext-channel attacks [12]; TDX adds MAC-protected memory; all still require an honest guest kernel for workload integrity. LapEE addresses the replay gap with four layers:

(1) **Hardware XTS tweak.** Relocated ciphertext decrypts to garbage. This is a confidentiality-and-location guarantee from the memory controller itself, not a software mitigation.

(2) **BEAM immutability.** User-space BEAM state is overwhelmingly immutable: once a term occupies an address, that address is not rewritten during its lifetime, so replay at an unchanged address is a no-op. The residual case is address reuse after GC: if A held C_1 for term P_1 , was freed, and reallocated for P_2 with C_2 , overwriting with C_1 yields P_1 on read. For exploit, P_1 must be structurally valid at P_2 ’s semantic slot — typically false, because BEAM tag bits and internal invariants will not match arbitrary stale bytes, and a mismatch surfaces as memory corruption rather than a usable term. Probabilistic.

(3) **Allocator randomization.** Randomize placement within allocator carriers to further reduce semantically-coherent address-reuse probability. Probabilistic.

(4) **TPM-backed counters for load-bearing invariants.** Where anti-replay must be cryptographic, `~tpm@2.0a` exposes volatile PCR extend (resets on reboot, for within-session freshness) and NV counter with PolicyPCR (monotonic across reboots; ~ 10 ms/increment; incrementable only when the golden PCR set is satisfied; for durable single-assignment invariants such as AO-Core slot heights).

Bulk state is defended probabilistically by (2)+(3); security-critical state by (4); all of it by (1) against relocation. (2)+(3) cover BEAM user-space terms only; kernel data, page tables, DMA rings, allocator metadata, and native-code buffers fall to the broader active-DRAM-write concern. The node signing key itself lives in the TPM, not DRAM. The *plaintext inputs* to sign operations do pass through DRAM, but an adversary tampering with them would corrupt the AO-Core hashpath tip — which is itself a merkle commitment to prior events — producing an inconsistency visible to the computation’s recipient. The most a user-space replay can achieve is re-signing of an input already signed; under RSASSA-PSS with fresh per-signature randomness the bytes differ but commit to the same message, compromising no downstream property we have identified. Active-write adversaries on DDR4/DDR5 require lab-grade bus interposers of the same cost-and-complexity class as the chip-decap attacks all deployed TEEs accept as out of scope.

7 Residual Threats

Physical attacks on the CPU or TPM package (decap, fault injection, laser, FIB) are accepted as a hardware-attack-complexity bar, consistent with TDX, SEV-SNP, and Apple PCC. Kernel 0-days (A1) are mitigated by signed image updates, a verifier-enforced OS version floor, and minimal userspace. Compromise of a trusted signing party is bounded by multi-party signing, reproducible builds, and public rotation logs. TME/SME key side channels have no published full break to date; the primitive is simpler than SGX’s MEE and is accordingly less studied. Rowhammer [18] is addressed by requiring DDR4 with targeted refresh or DDR5 with on-die ECC, recorded in attestation. Liveness and traffic analysis are out of scope; the former is an availability concern for the network layer, the latter requires an orthogonal padding or mixing scheme.

8 Hardware Availability

Class	Silicon capability	BIOS exposure
Intel vPro (client, 11th gen+)	TME since Tiger Lake 2020	~70–80%
Intel non-vPro (client, 11th gen+)	TME capable, patchy	~20–30%
Intel Xeon SP	TME since Ice Lake-SP 2021	~95%
AMD Ryzen Pro	TSME since Zen 2 2019	~70–80%
AMD Ryzen consumer	TSME capable, patchy	~20–40%
AMD EPYC	SME/TSME since Naples 2017	~95%
TPM 2.0	Win 11 requirement 2021	~100%
IOMMU	Business-class $\geq$ 2012	~100% bc
UEFI Secure Boot (enrollable PK)	Win 8 era 2012	~95%

Table 3: Primitive availability (engineering estimates; see methodology note below).

Per-feature composition (product of rates) yields roughly 11–35% viability on random consumer laptop purchases, rising above ~70% when filtered to the business-class secondary market (ThinkPad T/X/P, Latitude, EliteBook) where the per-feature rates cluster above ~90% conditional on the filter. Contrast: TDX-capable silicon (Sapphire Rapids+) is low-single-digit percent of the x86 server installed base; SEV-SNP similar; client CVM TEEs do not meaningfully exist. LapEE addresses an installed base roughly $100\times$ larger at roughly $50\text{--}100\times$ lower per-machine cost, often including zero-cost hardware already in operator possession.

Methodology. Percentages above are informed engineering estimates, not market-survey data. Sources: (a) Windows 11 system requirements (Microsoft) for TPM 2.0, IOMMU, Secure Boot as near-universal on Windows-11-era business hardware; (b) Intel product briefs for Tiger Lake+ and AMD datasheets for Zen 2+ for silicon capability; (c) Intel vPro and AMD Pro SKU feature disclosures for BIOS exposure estimates in their respective segments; (d) community compatibility lists for non-Pro consumer SKUs; (e) server BIOS documentation and industry reporting for Xeon SP and EPYC. We do not maintain a SKU-level database or claim a formal survey; readers should treat the numbers as an engineering estimate for the feasibility argument.

9 Implementation, Related Work, Conclusion

Implementation. mkosi [19] builds a UKI signed by HyperBEAM-OS keys, with a hardened minimal-driver kernel and IMA [20]; a dm-verity-sealed rootfs containing Erlang, HyperBEAM, and minimal systemd units; a systemd-pcrlock [14] manifest of expected PCR values. The new HyperBEAM device `~tpm@2.0a` (naming convention per `~lua@5.3a`) wraps tpm2-tss [21], exposing quote, sign, PCR-extend, NV-read/increment, and event-log-read, with encrypted sessions by default. Operator bring-up: enroll PK/KEK/db via `sbctl`; boot; HyperBEAM generates and PCR-extends its key, attests, registers.

Related work. Apple PCC [13] is architectural precedent for the minimal-attested-appliance design but assumes Apple-owned hardware in an Apple-controlled facility. Naik [5] attempts similar on Apple Silicon; it is defeated by the absence of a third-party- accessible measurement register, leaving the running-binary field of its attestation self-reported. Keylime [22] implements verifier-side policy evaluation; RATS [23] standardizes the terminology; BuilderNet [24] uses TDX with cloud-operator-set measurements — the circular trust LapEE avoids by operator-enrolled Secure Boot and open publication of golden measurements against reproducible source. Multi-node coordination follows for free: AO-Core is already a hyper-parallel decentralized compute substrate [2].

Conclusion. For the decentralized-compute threat model, a classical hardware TEE is not required. An attested dedicated appliance built on commodity primitives, running reproducibly-buildable code, trades only single-machine multi-tenant isolation — which the network does not need, because the network provides tenancy by distributing work. LapEE is a primitive; higher-layer predicates (`green-zone@1.0`’s `is-admissible`, consumer-side policies, blockchain-style managed- TEE schemes) compose on top. The ingredients exist now. They ship in ordinary business hardware. We have only to compose them.

Availability. Reproducible build recipes and the `~tpm@2.0a` device are being prepared at github.com/permaweb/hb-os.

References

- [1] Forward Research. *HyperBEAM: A decentralized node implementation for AO-Core*. 2024. URL: <https://github.com/permaweb/HyperBEAM>.
- [2] Sam Williams and Forward Research. *AO-Core: Hyper-Parallel Decentralized Computing on the Permaweb*. 2024. URL: <https://ao.ar.io/>.
- [3] Intel Corporation. *Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification*. 2023. URL: <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- [4] Advanced Micro Devices. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Whitepaper. 2020.
- [5] Gajesh Naik. *Private Decentralized Inference on Consumer Hardware: Enabling Confidential Computation on Third-Party Apple Silicon via Software Access Path Elimination*. Tech. rep. Eigen Labs, 2026.
- [6] Trusted Computing Group. *TPM 2.0 Library Specification, Family “2.0”, Level 00, Revision 01.59*. 2019. URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [7] UEFI Forum. *Unified Extensible Firmware Interface (UEFI) Specification, Version 2.10*. 2022. URL: <https://uefi.org/specifications>.
- [8] Intel Corporation. *Intel Architecture Memory Encryption Technologies Specification, Rev 1.4*. 2022. URL: <https://cdrdv2.intel.com/v1/dl/getContent/679154>.
- [9] David Kaplan, Jeremy Powell, and Tom Woller. *AMD Memory Encryption*. AMD Whitepaper. 2016. URL: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/memory-encryption-white-paper.pdf>.
- [10] Victor Costan and Srinivas Devadas. “Intel SGX Explained”. In: *IACR Cryptology ePrint Archive, Report 2016/086*. 2016.
- [11] Jo Van Bulck et al. “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *USENIX Security*. 2018.
- [12] Mengyuan Li et al. “CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel”. In: *USENIX Security*. 2021.
- [13] Apple Inc. *Blog: Private Cloud Compute: A new frontier for AI privacy in the cloud*. 2024. URL: <https://security.apple.com/blog/private-cloud-compute/>.
- [14] Lennart Poettering. *systemd-peclock(8) and “Brave New Trusted Boot World”*. 2023. URL: <https://0pointer.net/blog/brave-new-trusted-boot-world.html>.
- [15] The Linux Kernel Organization. *dm-verity: Device-mapper block integrity checking target*. 2024. URL: <https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/verity.html>.
- [16] The Linux Kernel Organization. *Kernel lockdown: confidentiality and integrity modes*. URL: <https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/index.html>.

- [17] Trusted Computing Group. *TCG PC Client Platform Firmware Profile Specification, Version 1.05 Revision 23*. 2021. URL: <https://trustedcomputinggroup.org/resource/pc-client-specific-platform-firmware-profile-specification/>.
- [18] Yoongu Kim et al. “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors”. In: *ISCA*. 2014.
- [19] systemd Project. *mkosi — Build Bespoke OS Images*. 2024. URL: <https://github.com/systemd/mkosi>.
- [20] The Linux Kernel Organization. *Integrity Measurement Architecture (IMA)*. 2024. URL: <https://sourceforge.net/p/linux-ima/wiki/Home/>.
- [21] TPM2 Software Community. *tpm2-tss: OSS implementation of the TCG TPM2 Software Stack*. 2024. URL: <https://github.com/tpm2-software/tpm2-tss>.
- [22] Nabil Schear et al. “Bootstrapping and Maintaining Trust in the Cloud”. In: *ACSAC*. 2016.
- [23] H. Birkholz et al. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, IETF. 2023. URL: <https://datatracker.ietf.org/doc/rfc9334/>.
- [24] Flashbots. *BuilderNet: a Decentralised Block Building Network for Ethereum*. 2024. URL: <https://buildernet.org/>.